PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS

Engenharia de Software

Pedro Henrique Maia Alves
Matrícula: 848301

DETECÇÃO DE BAD SMELLS E REFATORAÇÃO SEGURA

Belo Horizonte

2026

1. Análise de Bad Smells

A análise manual e o ESLint com o plugin SonarJS mostraram que o método generateReport concentra decisões de acesso, formatação e cálculo. Essa concentração dificulta mudanças isoladas e aumenta a quantidade de caminhos que precisam ser verificados pelos testes.

1.1 Método longo e complexidade cognitiva

O método original cuida do cabeçalho, filtra itens por papel, aplica prioridade, formata dois tipos de relatório e calcula o total. O SonarJS mediu complexidade cognitiva 27, acima do limite 5 configurado. A leitura exige acompanhar muitos níveis de decisão, o que torna alterações mais arriscadas e favorece regressões.

1.2 Duplicação de lógica

A geração das linhas CSV e HTML aparece em ramos separados para ADMIN e USER. O incremento do total também se repete. Se uma coluna ou regra de soma mudar, vários trechos precisam ser atualizados de forma idêntica; um esquecimento pode produzir relatórios inconsistentes e exigir testes redundantes.

1.3 Condicionais aninhadas

As verificações de papel, limite de valor e formato estão aninhadas dentro do mesmo laço. O ESLint identificou uma condicional colapsável na regra do usuário comum. Além de esconder a intenção da regra, essa estrutura multiplica os caminhos de execução e dificulta testar cada responsabilidade separadamente.

2. Relatório da Ferramenta

A execução anterior à refatoração confirmou dois erros. A análise automática quantificou a complexidade e apontou a estrutura condicional específica, complementando a inspeção manual.

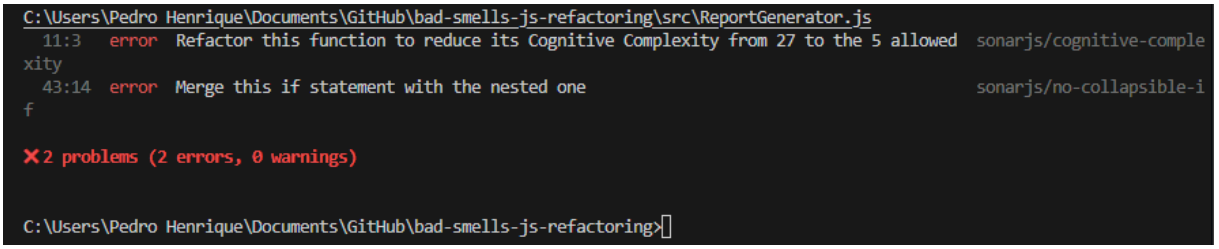


Figura 1 - Saída real do ESLint antes da refatoração

3. Processo de Refatoração

O smell mais crítico era a complexidade do método generateReport. A refatoração manteve a classe e sua interface pública, mas deslocou decisões específicas para funções e formatadores com uma única responsabilidade.

Antes	Depois

`for (const item of items) {`
`  if (user.role === 'ADMIN') {`
`  if (item.value > 1000) {`
`  item.priority = true;`
`  }`
`  if (reportType === 'CSV') {`
`  // gera CSV e soma o total`
`  } else if (reportType === 'HTML') {`
`  // gera HTML e soma o total`
`  }`
`  } else if (user.role === 'USER') {`
`  if (item.value <= 500) {`
`  // repete a decisão por formato`
`  }`
`  }`
`}`
	`const formatter =`
	`  REPORT_FORMATTERS[reportType];`
	
	`const visibleItems =`
	`  getVisibleItems(user, items);`
	
	`const total = visibleItems.reduce(`
	`  (sum, item) => sum + item.value,`
	`  0,`
	`);`
	
	`const body = visibleItems`
	`  .map((item) =>`
	`  formatter.item(item, user))`
	`  .join('');`
	
	`return `${formatter.header(user)}${body}` +`
	`  formatter.footer(total);`

Quadro 1 - Comparação do fluxo principal antes e depois

3.1 Técnicas aplicadas

- Extract Method separou a seleção dos itens visíveis na função getVisibleItems.
- Decompose Conditional transformou as regras de ADMIN e USER em retornos simples e nomeados.
- Os formatadores CSV e HTML foram reunidos em REPORT_FORMATTERS, eliminando a repetição entre papéis.
- Os valores 500 e 1000 e as strings de papel e formato foram substituídos por constantes com significado explícito.

A prioridade passou a ser calculada durante a formatação, sem alterar os objetos recebidos. O método principal agora apenas seleciona o formatador, obtém os itens, calcula o total e monta o resultado.

4. Validação Final e Conclusão

Antes da comparação, foram necessários dois ajustes mínimos no projeto-base: habilitar módulos ES no Jest e corrigir a marcação HTML de <tr > para <tr>. Nenhum teste fornecido foi alterado. Depois desses ajustes, os cinco testes originais passaram e serviram como rede de segurança para a refatoração.

Verificação	Código original	Código refatorado
Testes	5 de 5 aprovados	10 de 10 na suíte completa
ESLint	2 erros de smell	0 erros
Complexidade cognitiva	27	Abaixo do limite 5

5. Conclusão

Os testes permitiram alterar a estrutura interna sem perder o comportamento observado pelos usuários. A suíte confirmou a geração de CSV e HTML para administradores e usuários comuns, o filtro de itens, a prioridade visual e o tratamento da lista vazia. Quando uma correção preparatória mudou a saída HTML, o teste apontou exatamente a diferença, mostrando seu valor como rede de segurança.

A redução dos Bad Smells tornou as responsabilidades explícitas e diminuiu a complexidade medida pelo ESLint. O código refatorado pode receber novos formatos ou regras de visibilidade com menos alterações no fluxo principal. A análise estática encontrou estruturas difíceis de perceber apenas pela leitura, enquanto os testes provaram que a melhoria de design preservou os resultados esperados.

6. Referências.

FOWLER, Martin. Refatoração: Aperfeiçoando o Design de Códigos Existentes. 2. ed. São Paulo: Novatec, 2019.

SONARSOURCE. ESLint Plugin SonarJS. Disponível em: https://github.com/SonarSource/eslint-plugin-sonarjs.

FOWLER, Martin. Refactoring Catalog. Disponível em: https://refactoring.com/catalog/.